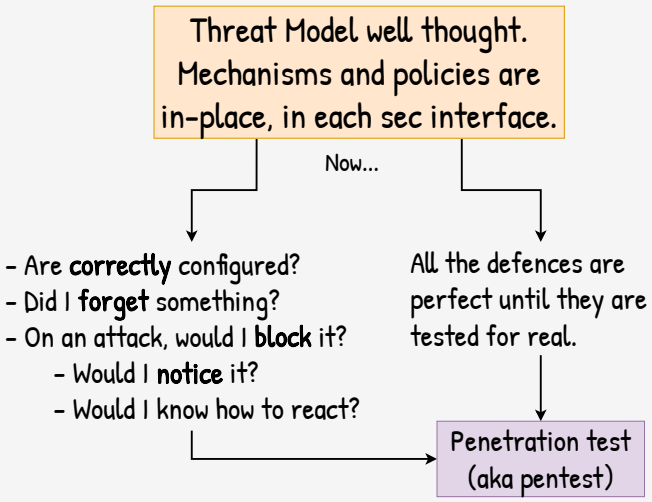
Pentesting - Offensive:
Red Team, Pentester & Researcher

Taller de Seguridad
Ing. Martin Di Paola - Fiuba



Red Team

Pentester

Researcher

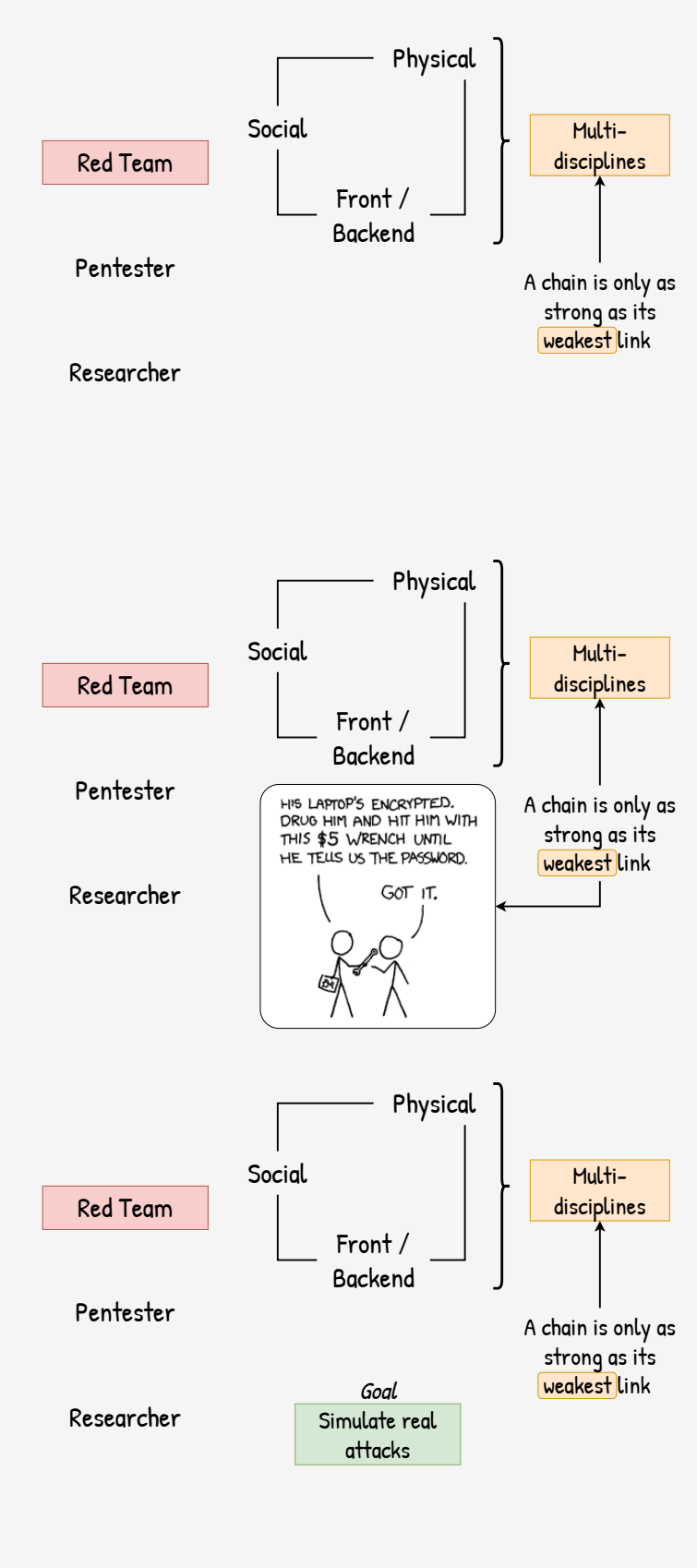
En el plano ofensivo tenemos a 3 players: Red Team, Pentester y Researcher (siendo el último el más alejado al campo de batalla)

No es una separacion tajante, hay mucha sinergia entre ellos:

- un red team tiene pentesters en su equipo

- los pentesters/red teams aprenden de las nuevas técni

- en general, tanto los miembros de un red team como lo



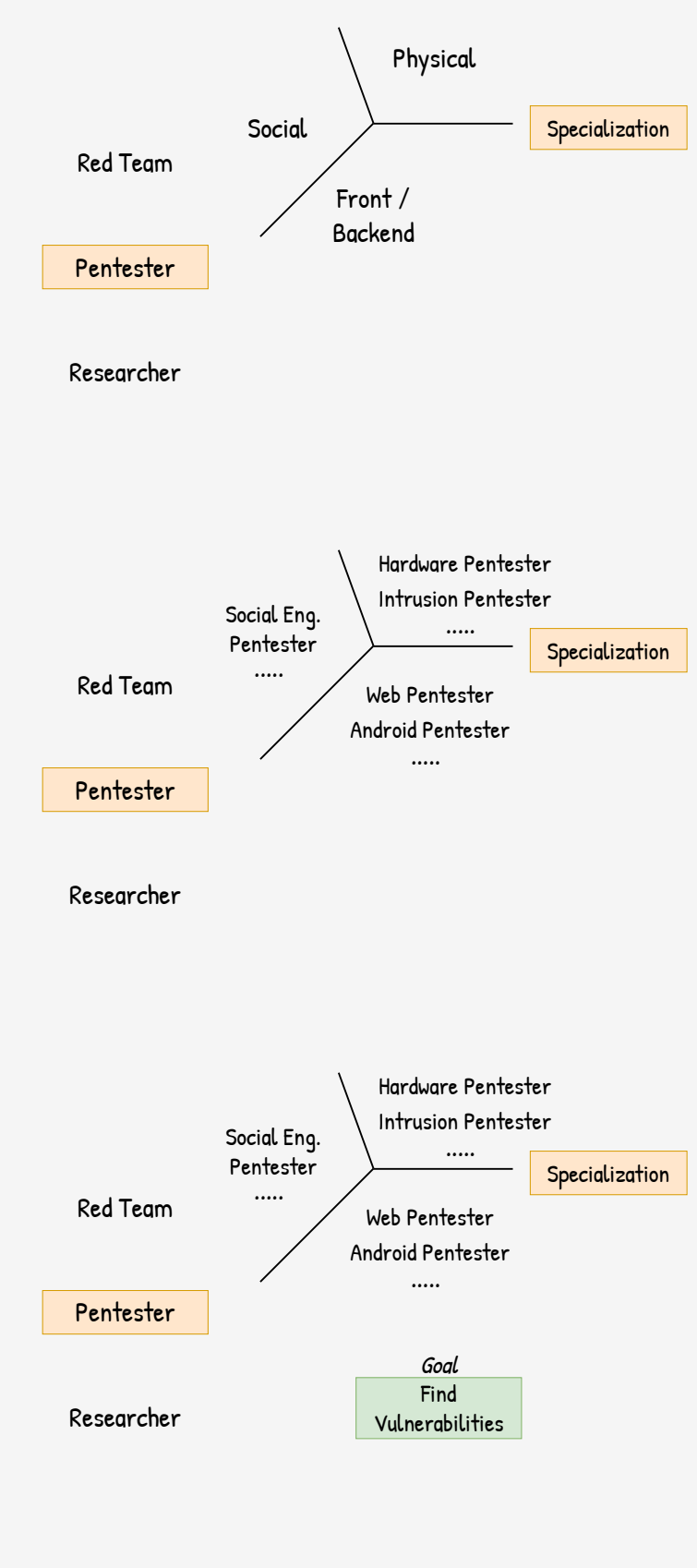
El Red Team es un equipo multidisciplinario que cubre todos los aspectos de seguridad: físicos, personas y software.

Pensa en un equipo que puede planear como entrar a un edificio lockpickeando las cerraduras, ir al datacenter abriendose paso con tarjetas de acceso clonadas y plantando software q vulnere a una empresa desde adentro.

Muy hardcore? Imaginete una campaña de investigación viendo los gustos personales del admin (IT) para enviarle un mail específicamente targeteado para que se instale un software y tomar control de su desktop, y de ahí, de toda la infra.

.

El objetivo es simular ataques reales, probar las defensas de una organización y ver que puntos flojos tiene tanto en los planos del software como en el hardware y en las personas.



Un pentester es una persona más especializada.

Los hay especialistas en apps web, apps de android, en hardware. Obviamente que cuanto mas hábil es el pentester más areas cubre.

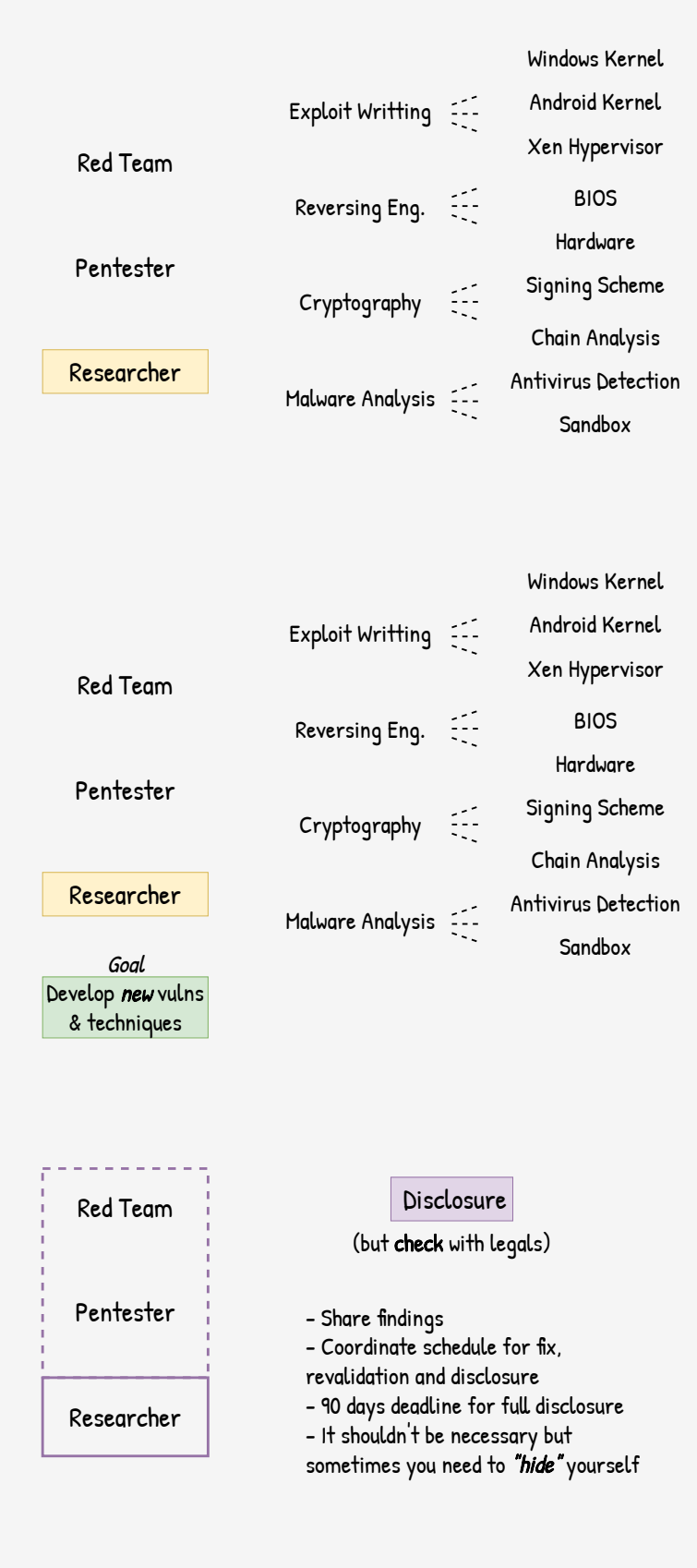
.

El objetivo es encontrar (y probar) vulnerabilidades pero sin explotarlas. Esto es, sin tomar ventaja de ellas.

No hay una diferencia clara de que es un Red Team y un Pentester.

Es fácil verla si uno compara un grupo de hackers capaces de entrar a un edificio, vulnerar las tarjetas de acceso y pawnear los datacenters y compara eso con una sola persona que solo revisa código de una app web.

Pero hay todo un abanico de opciones entre ambos extremos.



Los researchers (investigadores) son un bicho aparte: altamente especializados en un area, suelen estar lejos del campo de batalla.

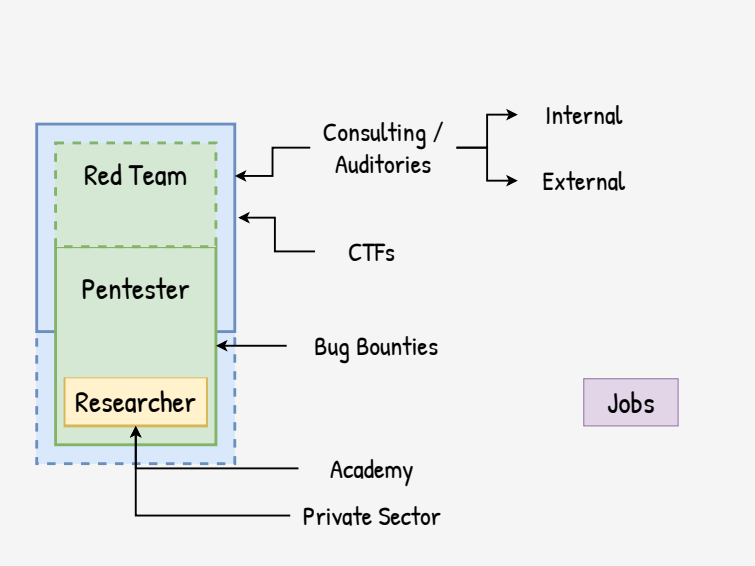
Se enfocan en encontrar vulnerabilidades y ataques nuevos.

A diferencia de un pentester que busca un SQLi en una app, un researcher busca nuevas formas de realizar un SQLi en general.

A todo esto, que se hace cuando se encuentra una vulnerabilidad?

Los red team/pentesters la reportan como parte de sus informes a sus clientes (cuando hay un contrato de por medio) o la reportan por una recompensa (bug bounty).

En general no hay forma de enforzar que toda vulnerabilidad sea reportada. Es la ética de la persona / empresa para la que trabaja.



Que motivación tiene cada uno?

Una empresa puede contratar los servicios de un red team / pentester para realizar una consultoría / auditoria (algunas empresas tienen equipos in-house, otras contratan equipos externos).

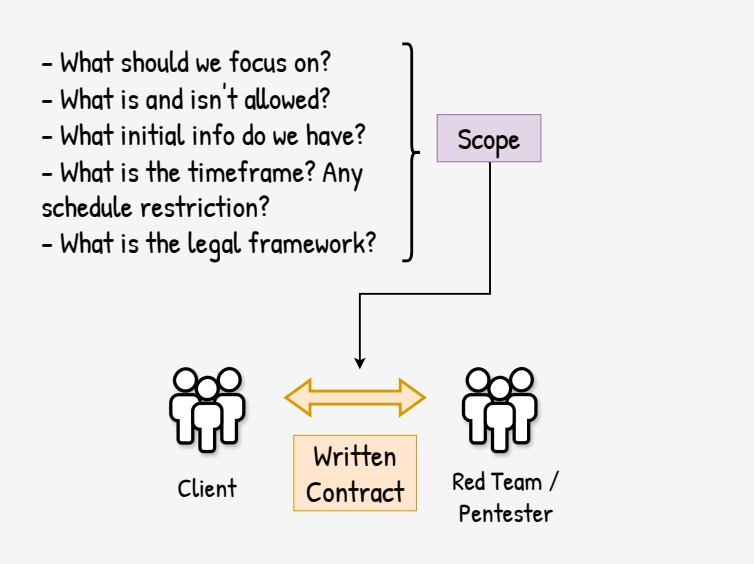
En otros casos, hay organizaciones que pagan por una recompensa a aquel que encuentra una vulnerabilidad en su software, los bug bounties.

Y los researchers?

Los resultados de investigación o bien se hacen públicos (cuando los researchers trabajan en la academia) o bien terminan siendo usados por empresas privadas para sus productos de seguridad.

Una empresa `X` cuyo producto es la automatización de ataques para pentesters/red team cuenta entre sus empleados researchers que encuentran nuevas vulns y ataques.

O, una empresa `Y` que ofrece logins de Windows con biometria, requiere de researchers para entender como funciona el login de Windows, hackearlo e integrar el login biometrico (por que Windows no ofrece forma ni documentación).



Veamos un ejemplo.

Tenemos un cliente que quiere realizar un pentest y contrata los servicios de un red team.

Que se quiere lograr? se quiere pawnear la DB? se quiere ver si es posible acceder físicamente al datacenter?

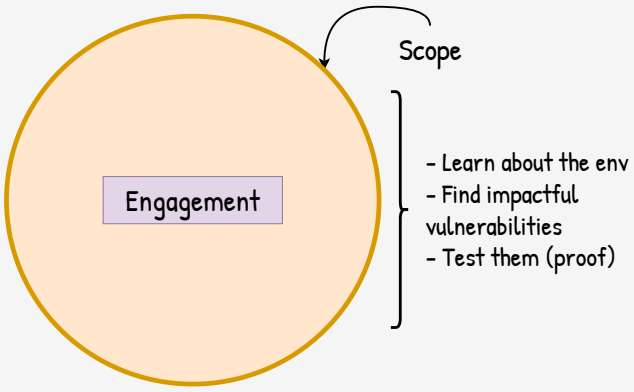
Que se permite y que no se permite hacer? podemos interactuar con los empleados? podemos explotar vulns sobre servicios en prod? aca hay que considerar los aspecto legales y pedir permisos explícitos.

Que información inicial tenemos?: en un modelo black box el red team arranca en cero siendo un escenario más realista para la simulación de un ataque.

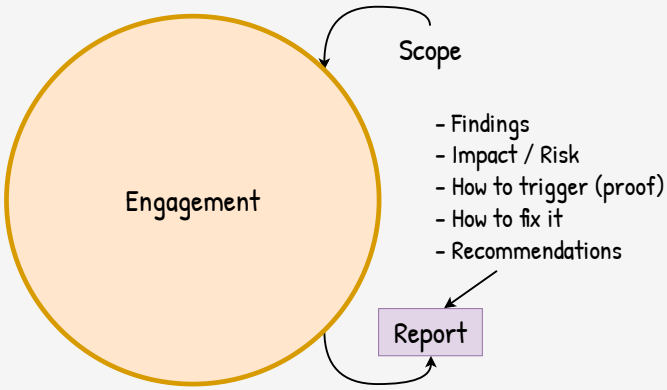
Sin embargo, idealmente se trata de convencer el cliente de ir por un modelo grey/white box con mas información sobre la infra para que el red team no pierda tanto tiempo buscando y se enfoque más en el ataque.

Cuanto tiempo se dispone? Hay otras restricciones? Si surge un problema, a quien se llama? (imaginate que el objetivo es entrar a un edificio pero te descubren y llaman a la policía, a quien llamas?!)

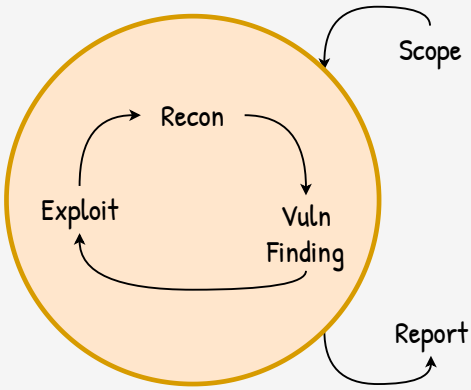
Todo esto forma parte del alcance del proyecto.



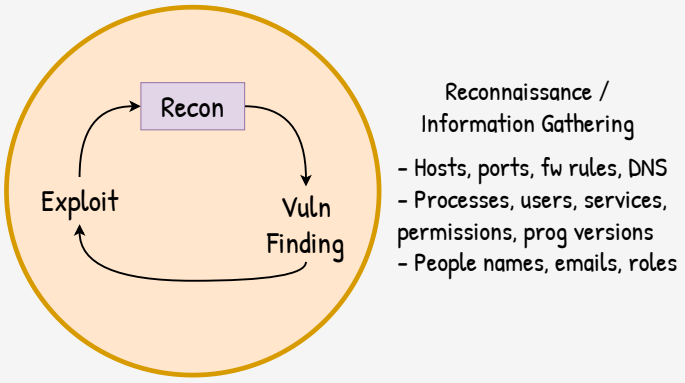
Definido el scope se pasa al engagement. Lease, la recolección de información, búsqueda de vulnerabilidades, planning y ataque.



Al finalizar, se arma un reporte. No solo le decimos al cliente que vulns encontramos sino también lo orientamos en como arreglarlas y priorizarlas.



El engagement es el core de un pentest: hay diferentes etapas pero a alto nivel se encuentran 3: recon/IG, find vulns y exploit.



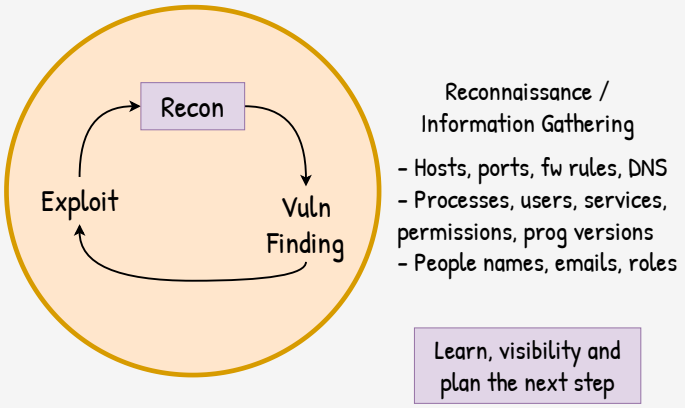
Information Gathering (IG) / Reconnaissance (recon)

Que hay? Que máquinas esta corriendo? Que puertos están abiertos? Que servicios, que versiones estan live? Que personas tienen acceso? Que roles hay?

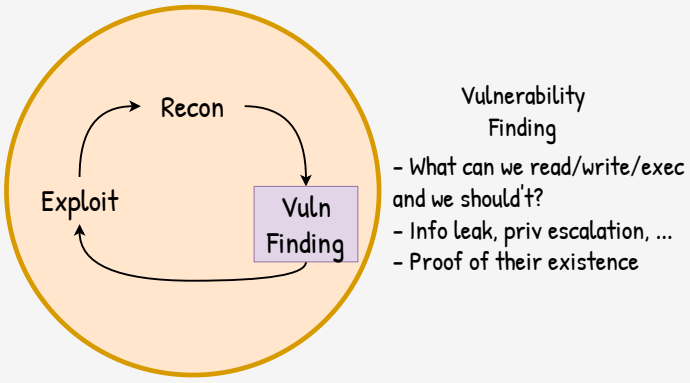
IG/Recon busca ver donde esta cada cosa y como se conectan las piezas.

En un modelo white box el cliente le puede suministrar al red team info de todo lo que hay en la infra, de tal forma que sea más sencillo planear los ataques.

En un modelo black box hay que buscar todo desde cero.



En resumen: tener visibilidad y saber que hay.

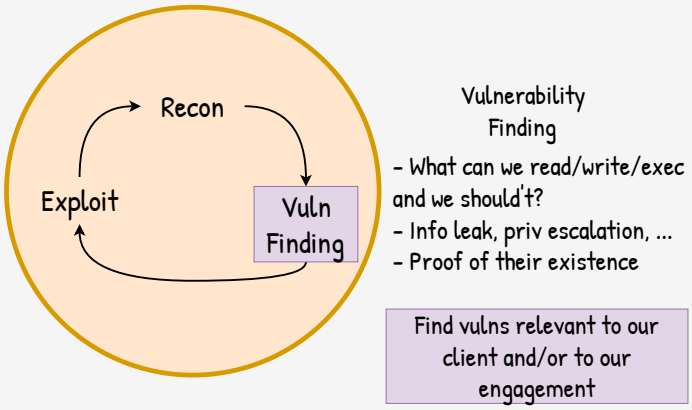


Find Vulns

De lo que se descubra en la etapa de recon se busca que cosas pueden ser vulnerables.

Hay máquinas accesibles por red que no debería estarlo? Hay puertos abiertos que no deberían estarlo? Hay servicios corriendo con versiones viejas vulnerables? Hay roles que tienen demasiados accesos?

Se busca que se puede leer / escribir / ejecutar que uno no debería.

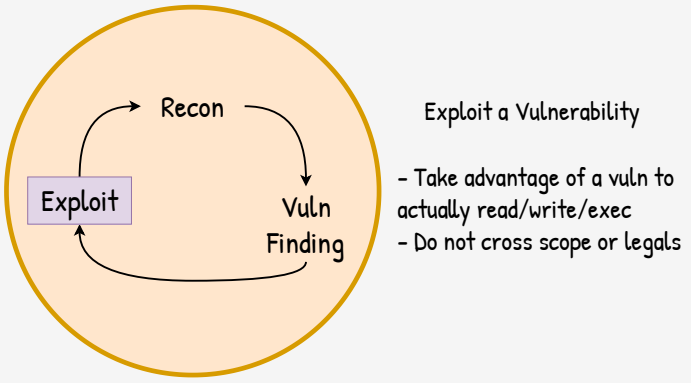


Se buscan todas las vulns posibles, pero en lo posible se orientan a aquellas que sean relevantes:

- una máquina interna respondió un ping? -> *bullshit*
- tiene el SSH abierto con credenciales por default? -> *that's a cookie!*

Se buscan vulns que nos permitan movernos dentro de la empresa: leer / escribir / ejecutar cosas que no deberíamos. Tener accesos que no deberíamos.

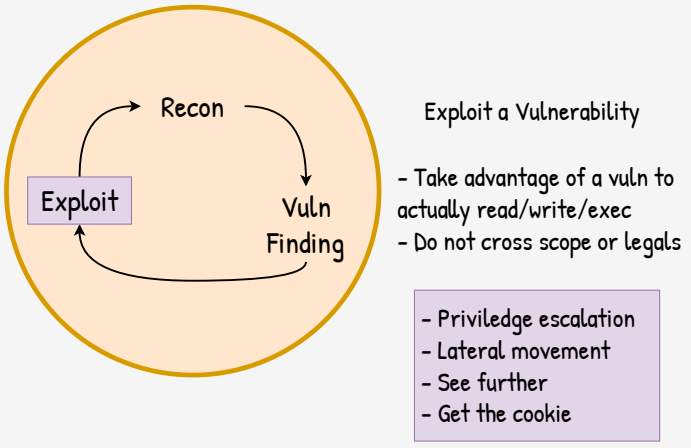
Esto es lo que habilita *avanzar y encontrar aun más cosas.*



Exploit!

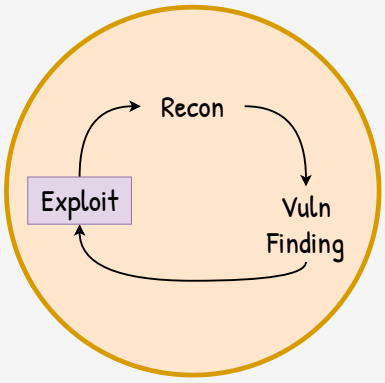
Es el uso de una vuln a nuestra ventaja.

No toda vuln es exploitable: una vulnerabilidad puede existir pero puede que no se pueda ejecutar o hacer algo útil con ella. La vuln se reporta igualmente pero no sirve para adquirir más accesos.



En que se traduce?

- En tener más privilegios
- Moverse lateralmente
- Poder ver más (habilitar mas IG/Recon)



Exploit a Vulnerability

- Take advantage of a vuln to actually read/write/exec
- Do not cross scope or legals

Enable **more** recon & vuln finding

El objetivo de explotar vulns es la de poder habilitar a mas recon y find vulns.

Atacar a un servicio remoto para forzarlo a ejecutar código arbitrario, instalarle un agente y obtener shell nos permite ahora realizar recon desde esa maquina, ver que vulns tiene esa maquina, que accesos podemos robar.

prod DB

pub.47

pub.48

Company's internal network

Scope:
- scan is OK
  - IPs: pub.47 pub.48
- use creds OK
  - but no creds given
- social eng **NOT** ok
- use exploits on prod **NOT** ok
- no more info (blackbox)

Goal:
- ~~exfiltrate prod DB~~
  - that's **NOT** for legals (us),
  - better: **prove** that prod DB can be exfiltrated

Supongamos que nuestro cliente nos dice:

- objetivo: filtrar la DB de producción
- que se puede/no se puede hacer: ...
- blackbox-model: solo te dire que hay 2 IPs públicas accesibles: la `.47` y la `.48`.

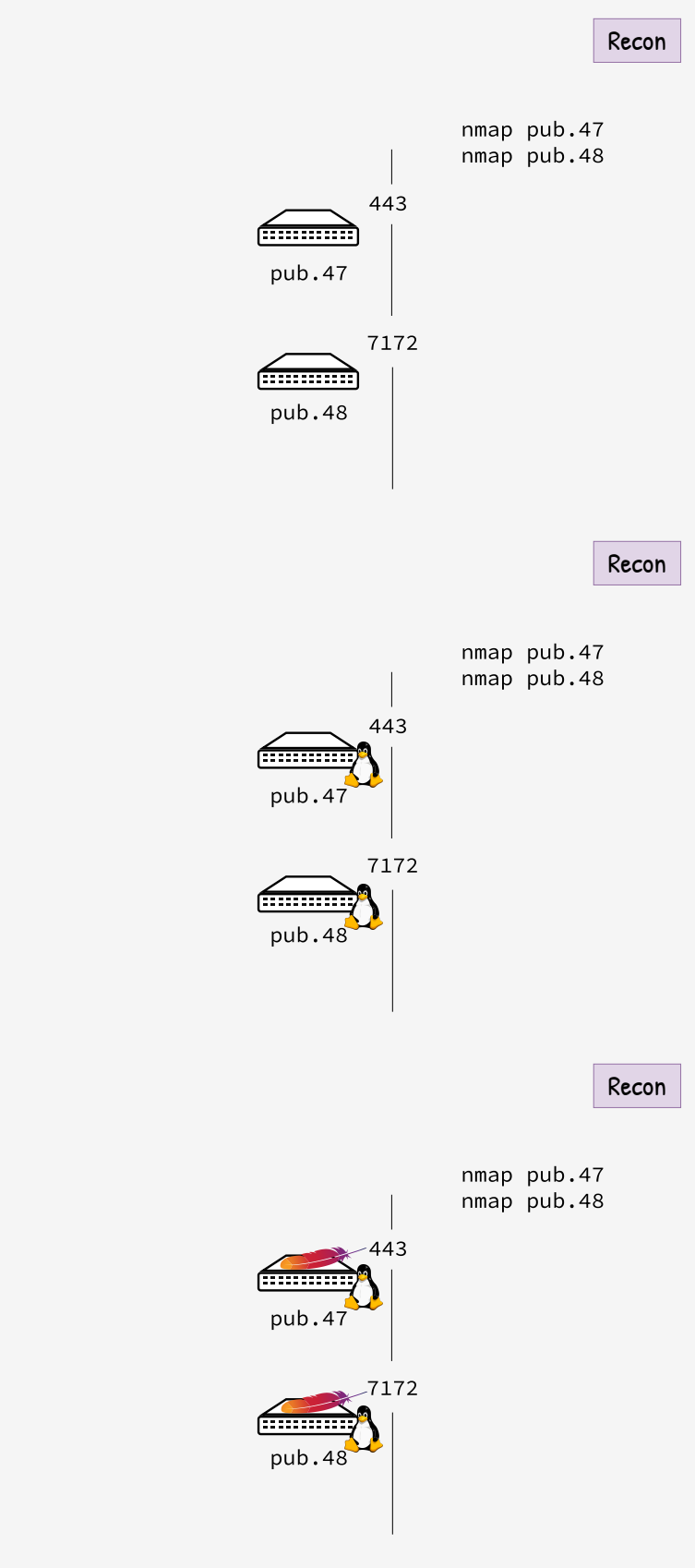
Si bien el objetivo es filtrar la DB de producción, si la DB contiene datos privados sensibles de usuarios reales podríamos estar incurriendo en un delito. Por lo tanto, el objetivo real va a ser *probarle* al cliente que se puede filtrar toda la DB sin hacerlo.

What's next?

Y que hacemos?

pub.47

pub.48

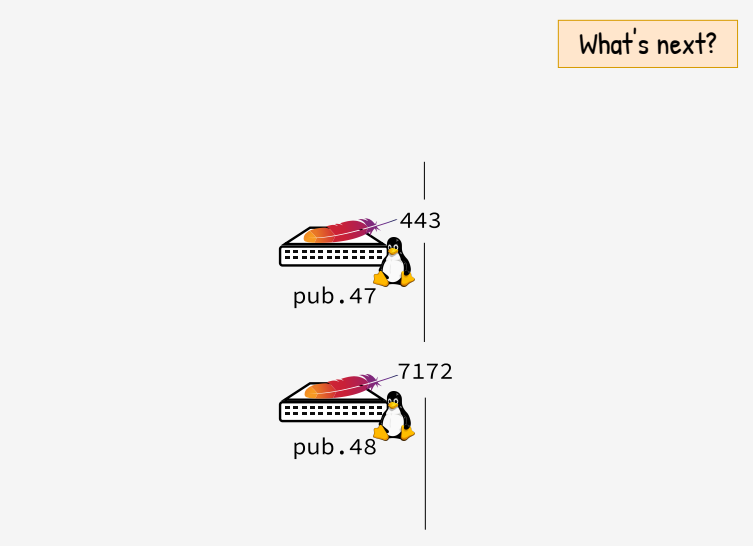


Recon!

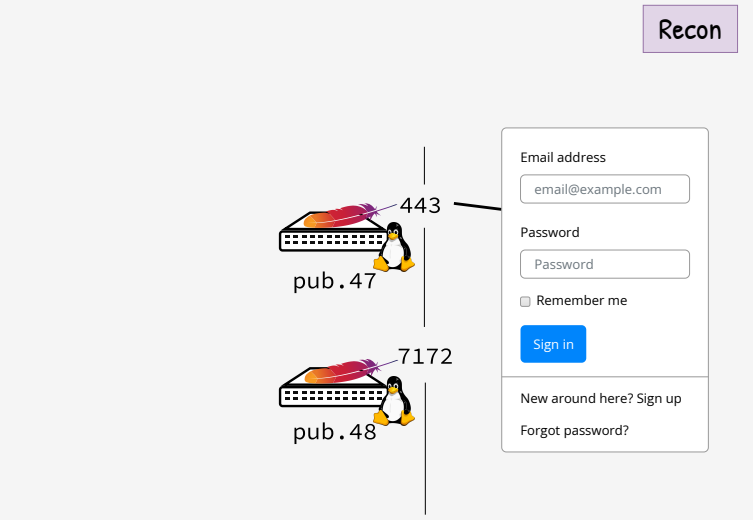
Podemos usar `nmap` para ver si los hosts y que puertos están activos.

Se puede inferir que OS están corriendo.

Y que servicios y que versiones estan corriendo.



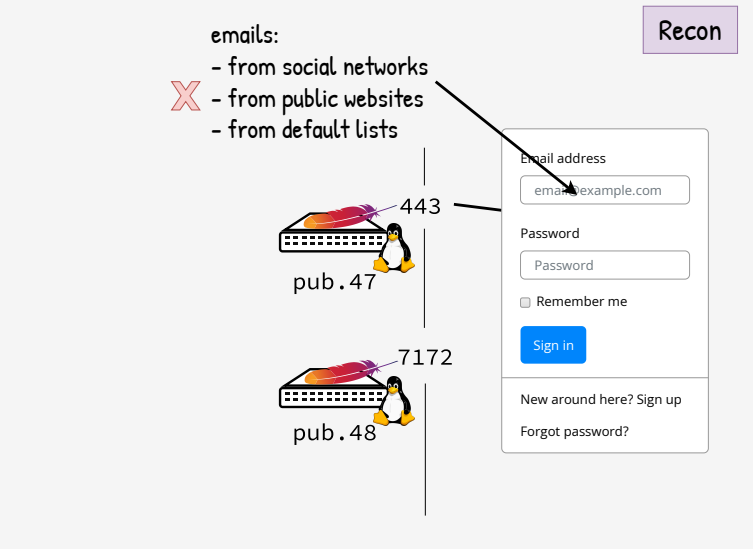
Y ahora, que hacemos?



Más Recon!

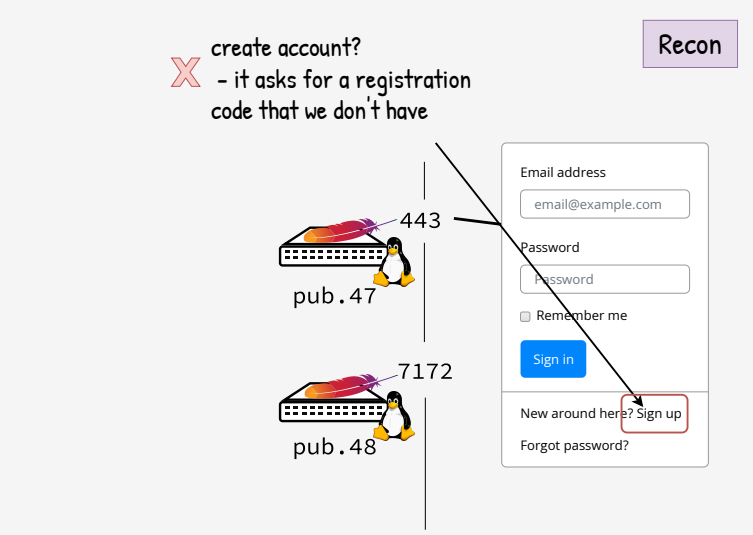
Si tenemos un server HTTP podríamos ver si podemos explotarlo pero este camino hay que descartarlo por que el cliente explícitamente nos dijo *"nada de atacar un servicio"*.

Entonces, solo podemos tratar de logguearnos y ver si hay algo mas detras.



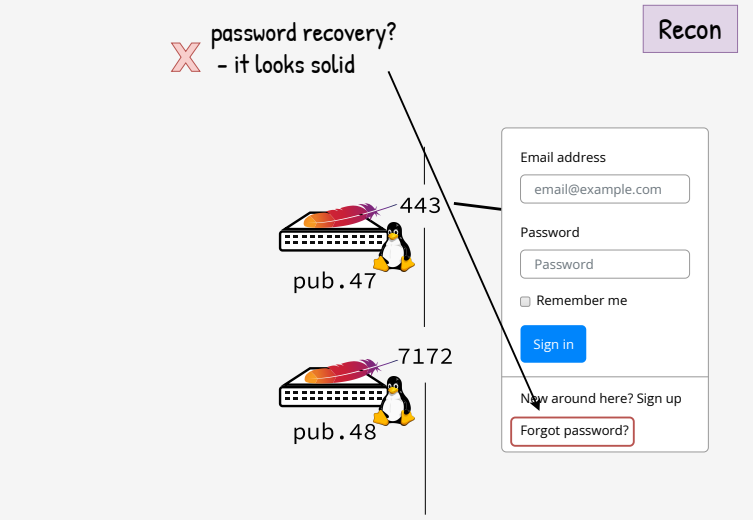
Probamos mails y passwords pero nada.

Podríamos tratar de engañar a un empleado para que nos de acceso, pero el cliente explícitamente nos dijo *"nada de ingeniería social"*.



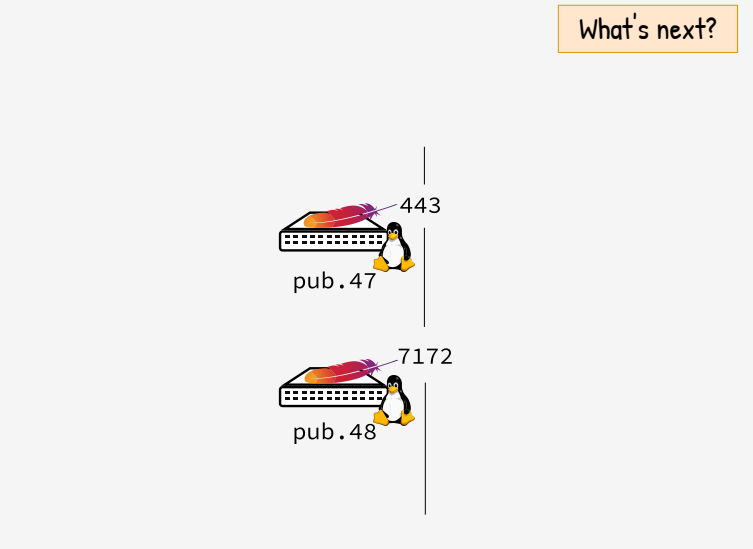
Tal vez podemos crearnos una cuenta?

Nope.

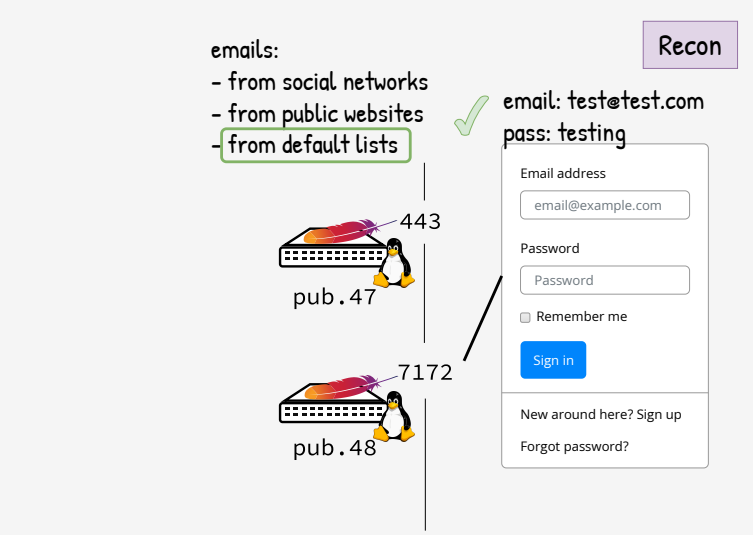


Tal vez podemos abusar del mecanismo de password recovery?

Nope.

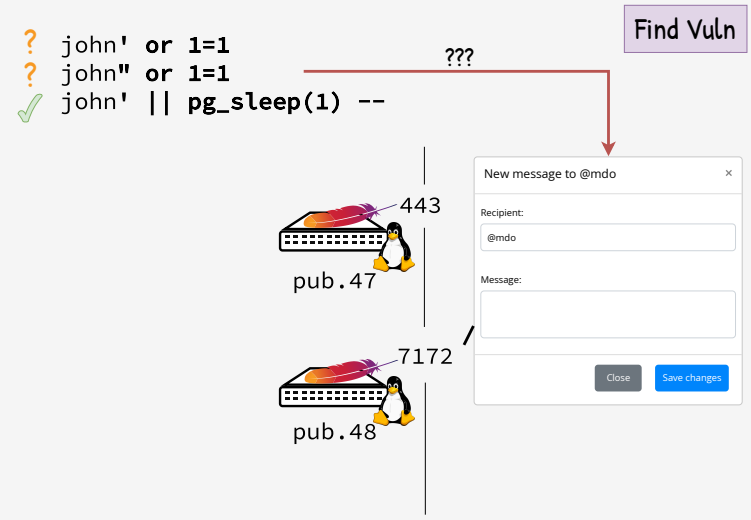


Y ahora, que hacemos?



En general, las cosas **no** son vulnerables. Por eso IG/Recon es importantísimo por que nos permite encontrar muchas más cosas con las que luego intentar romper.

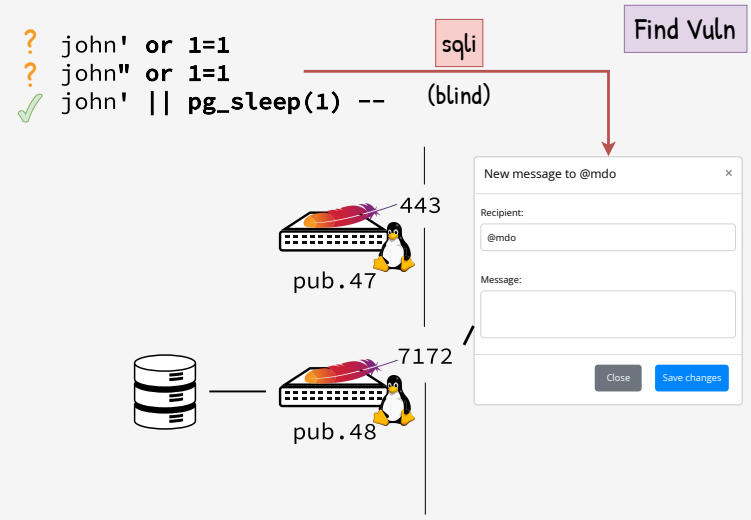
En la `.47` no tuvimos suerte, pero en la `.48` si!



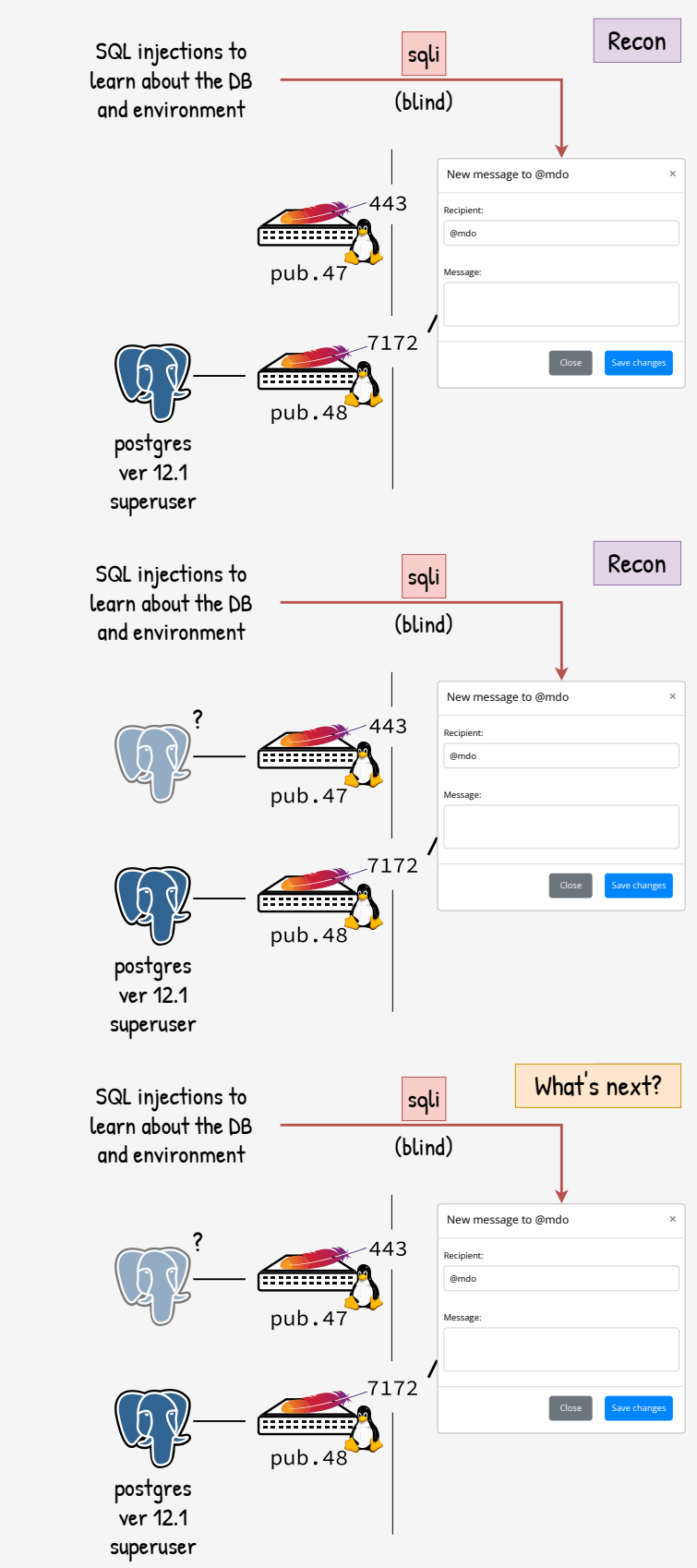
A encontrar vulnerabilidades!

Probando con `john' or 1=1` y similares, estamos:

- viendo si hay un SQL corriendo detras
- que el input es vulnerable a un SQL Injection (SQLi)
- y según como se comporte, hasta podemos saber que base de datos es



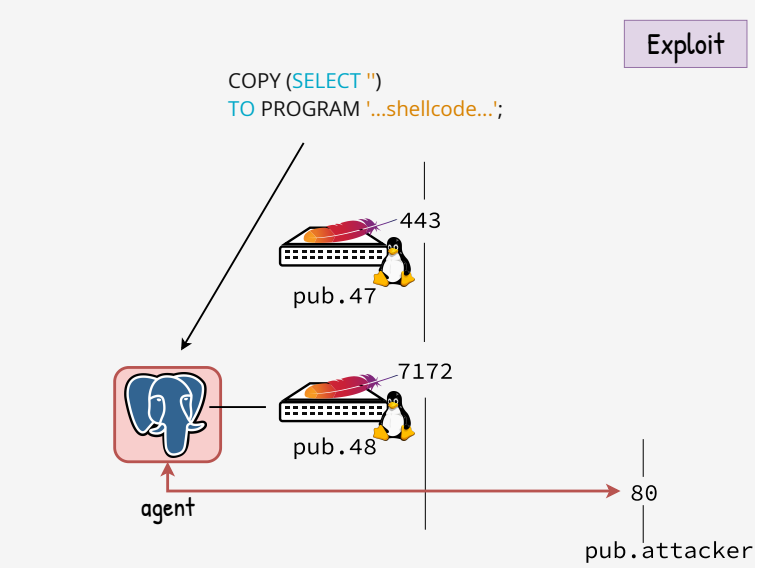
.



En un pentest estamos trabajando siempre con supuestos.

Si `.48` tiene usuarios de testing, es probable (aunque no confirmado) que la `.47` sea la máquina de producción.

Si la `.48` tiene un Postgres, la `.47` también.

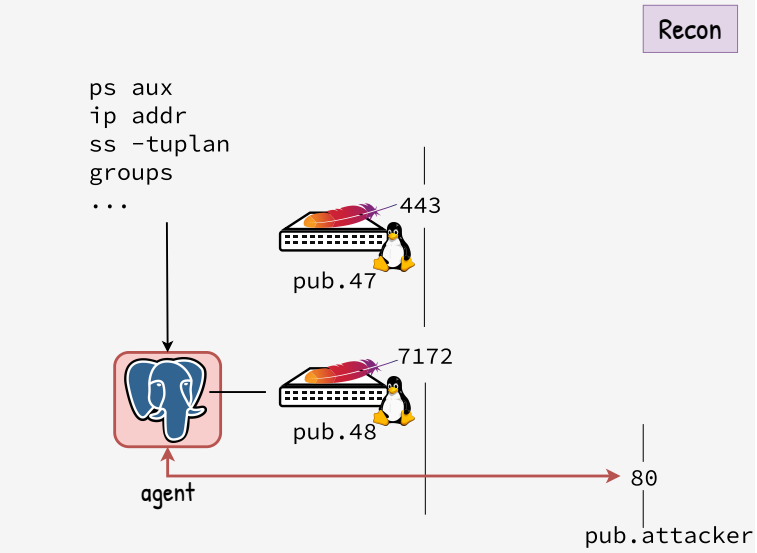


Exploit!

Inyectando algunas queries no solo podemos saber que hay Postgres sino también que version esta corriendo y con que usuario/role en la DB estamos ejecutándolas.

Con esa info podemos armar el exploit: una query que nos de un reverse-shell en la máquina que esta hosteando la DB.

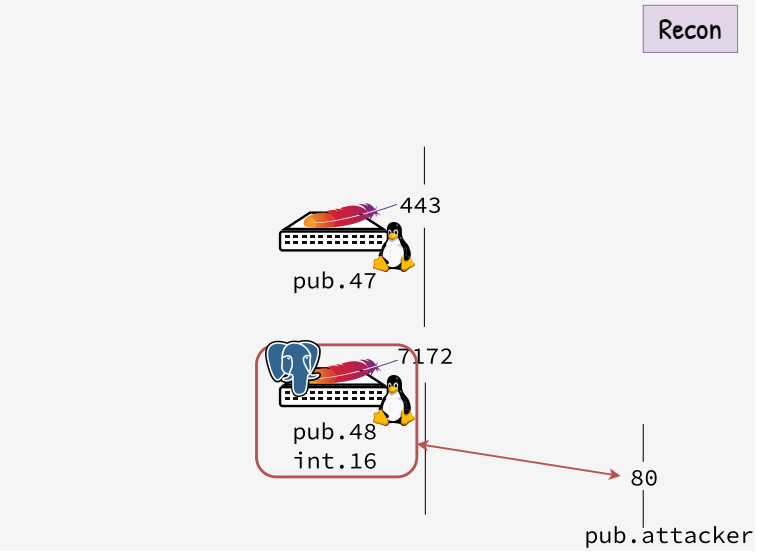
La idea es que se ejecute un programa en la máquina de la DB que se conecte hacia nuestros servidores para nosotros usar esa conexión y establecer un shell.



Y ahora?

Más IG/Recon!

Que procesos estan corriendo? Que usuarios hay? Que puertos estan en uso?



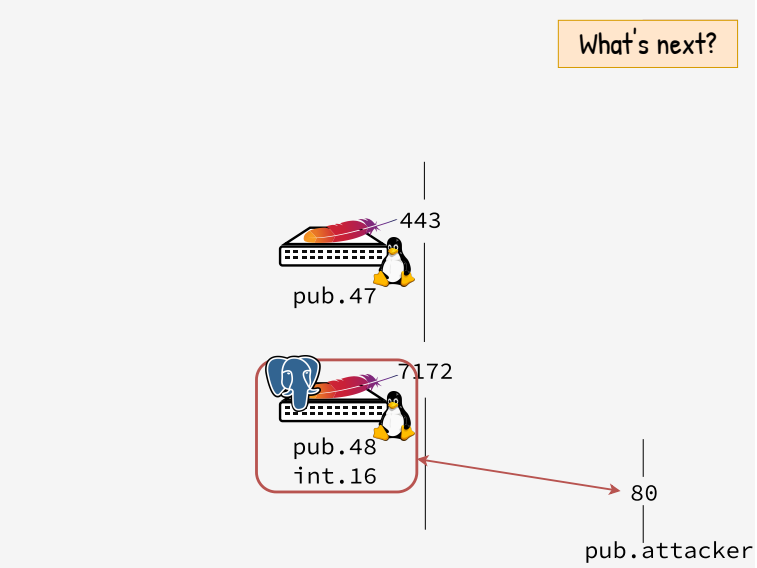
El IG/Recon nos dió que en la máquina de la DB hay un Apache y que esta siendo usado el mismo puerto `7172` y que la IP publica es la `.48`.

Conclusion?

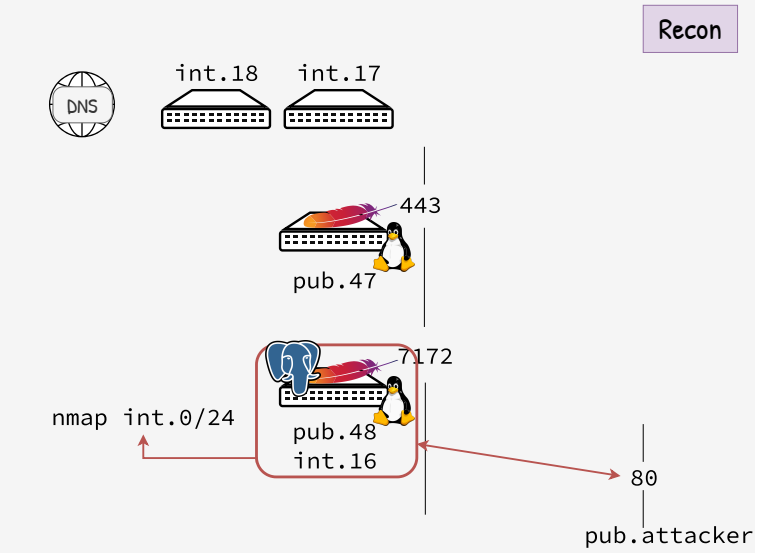
La máquina donde esta la DB y donde esta el Apache son la misma!

Y esto tiene sentido: en ambientes de testing muchas veces uno levanta toda la infra en una única máquina.

En un pentesting siempre se trabajan con supuestos y a medida que se avanza en el engage hay que ir cambiándolos.

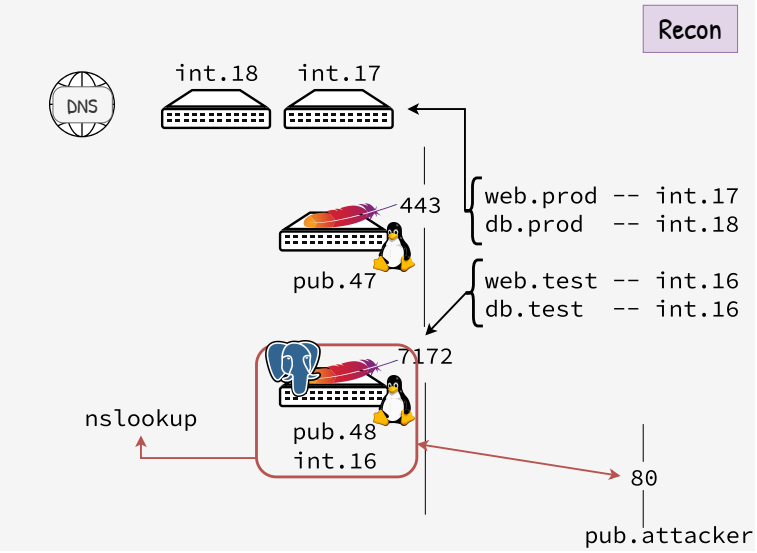


Y ahora, que hacemos?



Y ahora? Más IG/Recon.

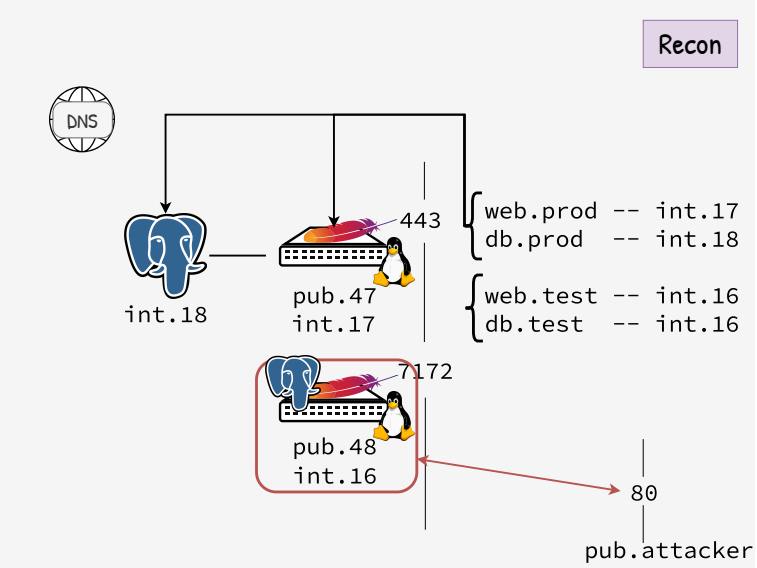
Ya desde adentro de la red del cliente, un `nmap` va a revelar cosas que desde afuera no estaban visibles.



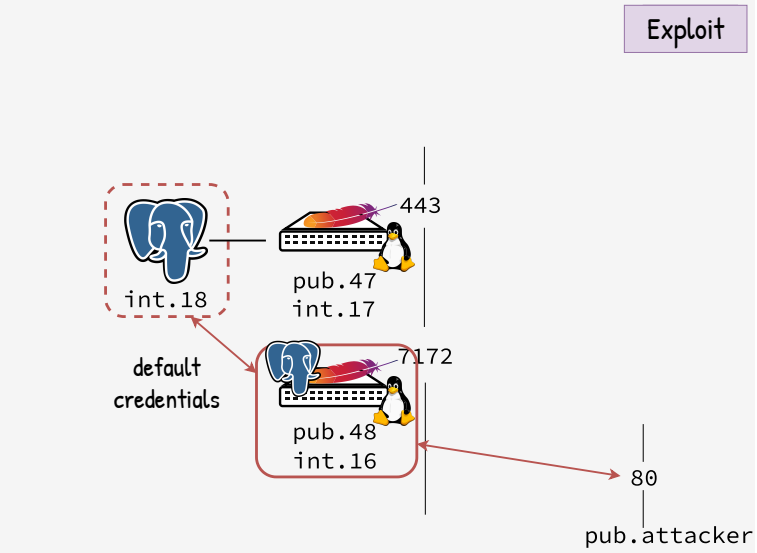
Y encontramos un DNS server.

Un DNS server no solo mapea hostnames a IPs sino, muchas veces, IPs a hostnames.

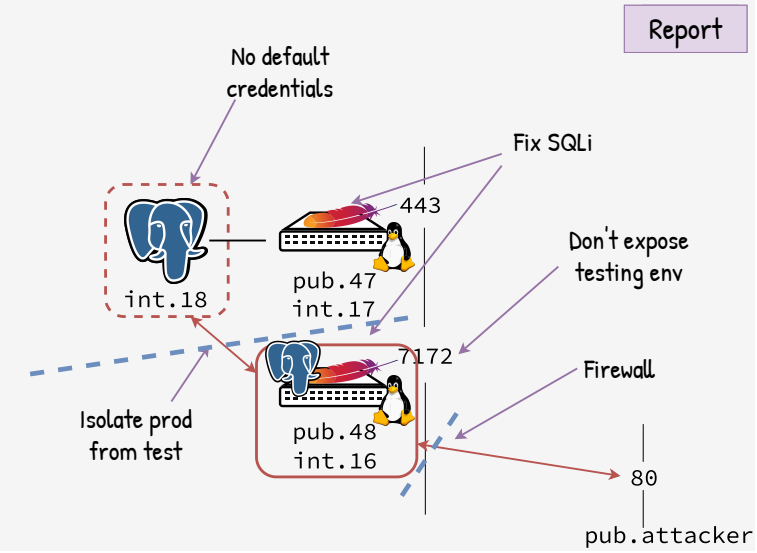
Que mejor forma de aprender que rol cumple cada máquina en la red que preguntándole a un DNS server! Un saludo al personal de IT por simplificarnos el trabajo!



.



Identificada la DB de producción, atacamos & profit.



El reporte a nuestro cliente no solo incluye el *attack path* o como logramos acceder y robarnos la DB sino también que otras vulns vimos en el camino y que medidas de seguridad deberían aplicarse.

No es solo bloquear una parte, es mejorar el todo: *sec in depth*.